

08-project

08. Final Project

Level: □ Advanced

Jam: 8 (4 minggu × 2 sesi)

Prasyarat: Semua modul sebelumnya (JS, TS, Express, DB, Frontend, AI)

Output: Full-stack AI-powered web app, live di internet, siap portfolio

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Merencanakan proyek dengan GitHub Project Board & ERD
- Menyiapkan project structure full-stack (Express + React/Next.js + Mastra AI + PostgreSQL)
- Membangun fitur CRUD dengan autentikasi pengguna
- Mengintegrasikan Mastra AI agent + tools ke dalam aplikasi
- Menangani error, loading state, dan responsive design
- Mendeploy aplikasi ke Vercel (frontend) + Railway (backend)
- Menulis README, dokumentasi API, dan presentasi proyek

Materi

Sesi	Topik	File
19-20	Sprint 1: Planning + Setup	01-sprint1.md
21-22	Sprint 2: Core Features (CRUD + Auth)	02-sprint2.md
23-24	Sprint 3: AI Integration	03-sprint3.md
25-26	Sprint 4: Polish + Deploy	04-sprint4.md

Output Akhir Modul

Full-stack AI-powered web app — deployed di Vercel + Railway, pakai Mastra AI agent dengan minimal 2 tools,

PostgreSQL database, autentikasi pengguna, dan siap dipresentasikan.

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- "Generate ERD for a travel planner app with users, trips, and destinations"
- "Create a PostgreSQL migration script for my schema"
- "Write Express middleware for JWT auth with refresh token"
- "Debug this API endpoint — it returns 500 on POST"
- "Design a Mastra tool that fetches weather data for itinerary"
- "Refactor this React component to use useReducer for loading states"
- "Generate a CI/CD pipeline for Vercel + Railway deploy"
- "Write a 5-minute presentation script for my final project"

Sprint 1 — Planning + Setup

Sesi: 19-20 | **Durasi:** 4 jam

Tujuan

- Brainstorm ide proyek dan tentukan fitur utama
- Buat GitHub repository + Project Board
- Gambar ERD (Entity Relationship Diagram)
- Setup project structure (monorepo / terpisah)
- Scaffold Express backend + database schema + migrasi
- Init Mastra AI agent
- Setup environment variables

Brainstorming & Project Ideas

Pilih salah satu proyek (atau modifikasi sesuai ide sendiri):

Project	Stack	Fitur AI
AI Travel Planner	TS + Express + Mastra + PG	Agent recommend itinerary + weather
Smart Notes App	TS + Express + Mastra + PG	AI summarize notes, auto-tag
Quiz Master	TS + Express + Mastra + PG	AI generate soal dari topic
Chat with Document	TS + Mastra + RAG + PG	Upload PDF, tanya jawab isinya

GitHub Project Board

Buat board dengan kolom: **Backlog** → **To Do** → **In Progress** → **Review** → **Done**

Contoh card:

- [Task] Setup Express + TypeScript
- [Task] Create database schema
- [Task] Init Mastra agent
- [Task] Design ERD
- [Task] Setup environment variables

ERD

Gambar ERD sesuai pilihan proyek. Contoh untuk Travel Planner:

users

id	UUID (PK)
email	string (unique)
password	string (hashed)
created_at	timestamp

trips

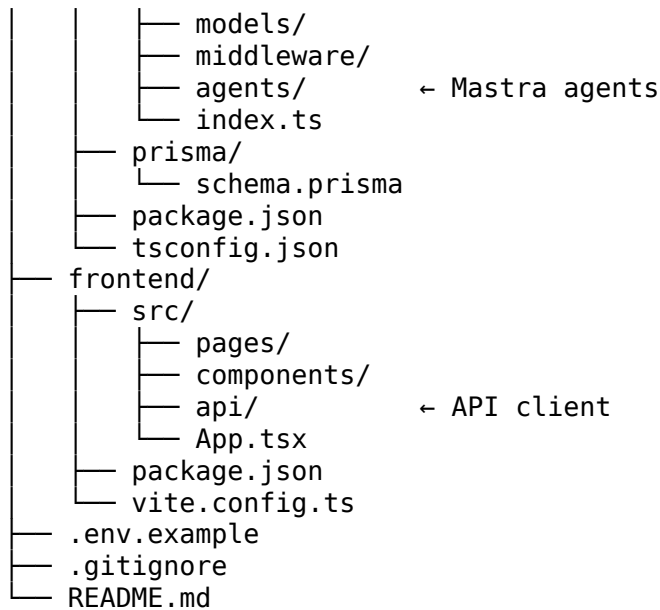
id	UUID (PK)
user_id	UUID (FK → users)
title	string
destination	string
start_date	date
end_date	date
created_at	timestamp

destinations

id	UUID (PK)
trip_id	UUID (FK → trips)
name	string
day	integer
notes	text
created_at	timestamp

Project Structure

```
my-project/  
├── backend/  
│   ├── src/  
│   │   ├── routes/  
│   │   └── controllers/
```



Express Scaffold

```
// backend/src/index.ts
import express from 'express';
import cors from 'cors';
import { PrismaClient } from '@prisma/client';

const app = express();
const prisma = new PrismaClient();
const PORT = process.env.PORT || 3001;

app.use(cors());
app.use(express.json());

app.get('/api/health', (_req, res) => {
  res.json({ status: 'ok' });
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Database Schema (Prisma)

```
// backend/prisma/schema.prisma
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model User {
  id          String   @id @default(uuid())
  email       String   @unique
  password    String
  createdAt   DateTime @default(now())
}
```

Mastra Agent Init

```
// backend/src/agents/travelAgent.ts
import { Agent } from '@mastra/core';

export const travelAgent = new Agent({
  name: 'travel-agent',
  instructions: `
    Kamu adalah asisten perjalanan yang membantu user merencanakan itinerary.
    Kamu bisa memberikan rekomendasi tempat wisata, akomodasi, dan transportasi.
    Gunakan tools yang tersedia untuk cek cuaca, cari tempat, dan simpan itinerary.
  `,
});
```

Environment Variables

```
# .env.example
DATABASE_URL=postgresql://user:pass@localhost:5432/myproject
PORT=3001
JWT_SECRET=mysecretkey
MASTRA_API_KEY=
OPENAI_API_KEY=
```

Latihan

1. **Bikin ERD** — Gambar ERD proyek kamu minimal 3 tabel (User + 2 entity lain) dengan relasi
2. **Setup GitHub** — Buat repo baru, inisialisasi project board dengan 5 task
3. **Scaffold Express** — Inisialisasi backend dengan Express + TypeScript + Prisma, pastikan `/api/health` return 200
4. **Database Migrate** — Jalankan `npx prisma migrate dev` sampai tabel User terbentuk di PostgreSQL
5. **Agent Test** — Init Mastra agent dengan instruksi sesuai project kamu, test panggil agent dari Node.js REPL

Sprint 2 — Core Features (CRUD + Auth)

Sesi: 21-22 | Durasi: 4 jam

Tujuan

- Bikin CRUD API endpoints untuk entity utama
- Bikin halaman frontend (list, form, detail)
- Integrasi API client di frontend
- Implementasi autentikasi (register/login/logout)
- Middleware JWT untuk proteksi route

CRUD API Endpoints

```
// backend/src/routes/trips.ts
import { Router, Request, Response } from 'express';
import { PrismaClient } from '@prisma/client';

const router = Router();
const prisma = new PrismaClient();

// GET /api/trips - list semua trips user
router.get('/', async (req: Request, res: Response) => {
  const trips = await prisma.trip.findMany({
    where: { userId: req.user.id },
  });
  res.json(trips);
});

// POST /api/trips - buat trip baru
router.post('/', async (req: Request, res: Response) => {
```

```

const { title, destination, startDate, endDate } = req.body;
const trip = await prisma.trip.create({
  data: {
    title,
    destination,
    startDate: new Date(startDate),
    endDate: new Date(endDate),
    userId: req.user.id,
  },
});
res.status(201).json(trip);
});

// GET /api/trips/:id - detail trip
router.get('/:id', async (req: Request, res: Response) => {
  const trip = await prisma.trip.findFirst({
    where: { id: req.params.id, userId: req.user.id },
  });
  if (!trip) return res.status(404).json({ error: 'Trip not found' });
  res.json(trip);
});

// PUT /api/trips/:id - update trip
router.put('/:id', async (req: Request, res: Response) => {
  const { title, destination, startDate, endDate } = req.body;
  const trip = await prisma.trip.update({
    where: { id: req.params.id },
    data: { title, destination, startDate, endDate },
  });
  res.json(trip);
});

// DELETE /api/trips/:id - hapus trip
router.delete('/:id', async (req: Request, res: Response) => {
  await prisma.trip.delete({ where: { id: req.params.id } });
  res.json({ message: 'Deleted' });
});

export default router;

```

Auth Middleware

```

// backend/src/middleware/auth.ts
import { Request, Response, NextFunction } from 'express';
import jwt from 'jsonwebtoken';

```

```

interface AuthRequest extends Request {
  user?: { id: string; email: string };
}

export const authMiddleware = (
  req: AuthRequest,
  res: Response,
  next: NextFunction
) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) {
    return res.status(401).json({ error: 'No token provided' });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET!) as {
      id: string;
      email: string;
    };
    req.user = decoded;
    next();
  } catch {
    res.status(401).json({ error: 'Invalid token' });
  }
};

```

Auth Controller

```

// backend/src/controllers/auth.ts
import { Request, Response } from 'express';
import bcrypt from 'bcrypt';
import jwt from 'jsonwebtoken';
import { PrismaClient } from '@prisma/client';

const prisma = new PrismaClient();

export const register = async (req: Request, res: Response) => {
  const { email, password } = req.body;

  const existing = await prisma.user.findUnique({ where: { email } });
  if (existing) {
    return res.status(400).json({ error: 'Email already registered' });
  }
}

```



```

const hashed = await bcrypt.hash(password, 10);
const user = await prisma.user.create({
  data: { email, password: hashed },
});

const token = jwt.sign(
  { id: user.id, email: user.email },
  process.env.JWT_SECRET!,
  { expiresIn: '7d' }
);

res.status(201).json({ token, user: { id: user.id, email: user.email } });
};

export const login = async (req: Request, res: Response) => {
  const { email, password } = req.body;

  const user = await prisma.user.findUnique({ where: { email } });
  if (!user) {
    return res.status(401).json({ error: 'Invalid credentials' });
  }

  const valid = await bcrypt.compare(password, user.password);
  if (!valid) {
    return res.status(401).json({ error: 'Invalid credentials' });
  }

  const token = jwt.sign(
    { id: user.id, email: user.email },
    process.env.JWT_SECRET!,
    { expiresIn: '7d' }
  );

  res.json({ token, user: { id: user.id, email: user.email } });
};

```

Frontend — Halaman List

```

// frontend/src/pages/TripList.tsx
import { useEffect, useState } from 'react';
import { api } from '../api/client';

interface Trip {
  id: string;
  title: string;
}

```

```

    destination: string;
    startDate: string;
  }

export default function TripList() {
  const [trips, setTrips] = useState<Trip[]>([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    api.get('/trips')
      .then((res) => setTrips(res.data))
      .finally(() => setLoading(false));
  }, []);

  if (loading) return <div>Loading...</div>;

  return (
    <div>
      <h1>My Trips</h1>
      {trips.map((trip) => (
        <div key={trip.id} className="card">
          <h2>{trip.title}</h2>
          <p>{trip.destination}</p>
          <p>{trip.startDate}</p>
        </div>
      ))}
    </div>
  );
}

```

Frontend — Form

```

// frontend/src/pages/TripForm.tsx
import { useState } from 'react';
import { api } from '../api/client';

export default function TripForm() {
  const [form, setForm] = useState({
    title: '',
    destination: '',
    startDate: '',
    endDate: '',
  });
  const [error, setError] = useState('');

```

```

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  try {
    await api.post('/trips', form);
    alert('Trip created!');
  } catch (err) {
    setError('Failed to create trip');
  }
};

return (
  <form onSubmit={handleSubmit}>
    {error && <div className="error">{error}</div>}
    <input
      placeholder="Title"
      value={form.title}
      onChange={(e) => setForm({ ...form, title: e.target.value })}
    />
    <input
      placeholder="Destination"
      value={form.destination}
      onChange={(e) => setForm({ ...form, destination: e.target.value })}
    />
    <input
      type="date"
      value={form.startDate}
      onChange={(e) => setForm({ ...form, startDate: e.target.value })}
    />
    <input
      type="date"
      value={form.endDate}
      onChange={(e) => setForm({ ...form, endDate: e.target.value })}
    />
    <button type="submit">Create Trip</button>
  </form>
);
}

```

API Client

```

// frontend/src/api/client.ts
import axios from 'axios';

const token = localStorage.getItem('token');

```

```
export const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || 'http://localhost:3001/api',
  headers: {
    Authorization: token ? `Bearer ${token}` : '',
  },
});
```

Latihan

1. **Bikin Endpoint CRUD** — Buat routes + controllers untuk entity kamu (trips/notes/quiz) minimal GET, POST, PUT, DELETE
2. **Bikin Halaman Form** — Buat komponen React form dengan validasi client-side
3. **Integrasi API** — Hubungkan frontend ke backend pakai axios, tampilkan data dari API
4. **Auth Flow** — Implement register → login → token disimpan di localStorage → dipake di setiap request
5. **Proteksi Route** — Tambah middleware auth ke endpoint, pastikan user yang belum login dapat 401

Sprint 3 — AI Integration

Sesi: 23-24 | Durasi: 4 jam

Tujuan

- Wiring Mastra agent ke aplikasi
- Bikin tools Mastra (weather, search, itinerary)
- Agent response ditampilkan di frontend
- Stream response dari agent
- Error handling + loading states
- Prompt templates terstruktur

Mastra Tool — Weather

```
// backend/src/agents/tools/weather.ts
import { Tool } from '@mastra/core';
```

```
interface WeatherInput {
  city: string;
  date: string;
}
```

```
export const weatherTool = new Tool<WeatherInput>({
```

```

name: 'get-weather',
description: 'Get weather forecast for a city on a specific date',
inputSchema: {
  type: 'object',
  properties: {
    city: { type: 'string', description: 'Nama kota' },
    date: { type: 'string', description: 'Tanggal (YYYY-MM-DD)' },
  },
  required: ['city'],
},
execute: async ({ input }) => {
  // Panggil OpenWeatherMap API atau mock
  const response = await fetch(
    `https://api.openweathermap.org/data/2.5/weather?q=${input.city}&appid=${p
  );
  const data = await response.json();
  return {
    temperature: data.main.temp,
    description: data.weather[0].description,
    humidity: data.main.humidity,
  };
},
});

```

Agent dengan Tools

```

// backend/src/agents/travelAgent.ts
import { Agent } from '@mastra/core';
import { weatherTool } from '../tools/weather';
import { searchPlaceTool } from '../tools/searchPlace';
import { saveItineraryTool } from '../tools/saveItinerary';

export const travelAgent = new Agent({
  name: 'travel-agent',
  instructions: `
    Kamu adalah asisten perjalanan yang membantu user merencanakan itinerary.

    Langkah-langkah:
    1. Tanya tujuan dan tanggal perjalanan
    2. Cek cuaca destinasi pake tool get-weather
    3. Cari tempat wisata populer pake tool search-place
    4. Simpan itinerary pake tool save-itinerary
    5. Tampilkan ringkasan rencana perjalanan

    Selalu gunakan tools yang tersedia – jangan cuma ngasih saran umum.
  `,
});

```

```

    \,
    tools: [weatherTool, searchPlaceTool, saveItineraryTool],
  });

```

Agent API Endpoint — Stream

```

// backend/src/routes/ai.ts
import { Router, Request, Response } from 'express';
import { travelAgent } from '../agents/travelAgent';

const router = Router();

// POST /api/ai/chat - chat dengan agent (streaming)
router.post('/chat', async (req: Request, res: Response) => {
  const { message } = req.body;

  res.setHeader('Content-Type', 'text/plain');
  res.setHeader('Transfer-Encoding', 'chunked');

  try {
    const stream = await travelAgent.stream([
      { role: 'user', content: message },
    ]);

    for await (const chunk of stream) {
      res.write(chunk.delta?.text || '');
    }

    res.end();
  } catch (error) {
    res.status(500).json({ error: 'AI agent error' });
  }
});

export default router;

```

Frontend — Chat Component

```

// frontend/src/components/Chat.tsx
import { useState, useRef } from 'react';

export default function Chat() {
  const [messages, setMessages] = useState<
    { role: 'user' | 'ai'; content: string }[]
  >([]);

```

```

const [input, setInput] = useState('');
const [loading, setLoading] = useState(false);
const [abortController, setAbortController] =
  useState<AbortController | null>(null);

const handleSend = async () => {
  const userMsg = { role: 'user' as const, content: input };
  setMessages((prev) => [...prev, userMsg]);
  setInput('');
  setLoading(true);

  const controller = new AbortController();
  setAbortController(controller);

  try {
    const res = await fetch(
      `${import.meta.env.VITE_API_URL}/ai/chat`,
      {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ message: input }),
        signal: controller.signal,
      }
    );

    const reader = res.body?.getReader();
    const decoder = new TextDecoder();

    let aiContent = '';
    setMessages((prev) => [...prev, { role: 'ai', content: '' }]);

    while (reader) {
      const { done, value } = await reader.read();
      if (done) break;

      const text = decoder.decode(value);
      aiContent += text;

      setMessages((prev) => {
        const updated = [...prev];
        updated[updated.length - 1] = {
          role: 'ai',
          content: aiContent,
        };
        return updated;
      });
    }
  }
};

```

```

    }
  } catch (err: any) {
    if (err.name !== 'AbortError') {
      setMessages((prev) => [
        ...prev,
        { role: 'ai', content: 'Error: gagal mendapat response' },
      ]);
    }
  } finally {
    setLoading(false);
    setAbortController(null);
  }
};

const handleStop = () => {
  abortController?.abort();
};

return (
  <div className="chat-container">
    <div className="messages">
      {messages.map((msg, i) => (
        <div key={i} className={`message ${msg.role}`}>
          <strong>{msg.role === 'user' ? 'You' : 'AI'}:</strong>{' '}
          {msg.content}
        </div>
      ))}
    </div>
    <div className="input-area">
      <input
        value={input}
        onChange={(e) => setInput(e.target.value)}
        onKeyDown={(e) => e.key === 'Enter' && handleSend()}
        disabled={loading}
        placeholder="Tanya rencana perjalanan..."
      />
      {loading ? (
        <button onClick={handleStop}>Stop</button>
      ) : (
        <button onClick={handleSend}>Send</button>
      )}
    </div>
  </div>
);
}

```


Prompt Templates

```
// backend/src/agents/prompts.ts
export const ITINERARY_PROMPT = `
Buatkan itinerary perjalanan ke {{destination}} dari {{startDate}} sampai {{endDate}}.

Pertimbangkan:
- Cuaca pada tanggal tersebut (gunakan tool get-weather)
- Tempat wisata populer (gunakan tool search-place)
- Budget dan durasi per hari

Output dalam format:
## Itinerary: {{destination}}
### Day 1 – {{date}}
- Pagi: ...
- Siang: ...
- Malam: ...
### Day 2 – {{date}}
...
`;

export const RECOMMENDATION_PROMPT = `
Berdasarkan preferensi user: {{preferences}}
Beri rekomendasi {{count}} tempat wisata di {{city}}.

Setiap rekomendasi harus punya:
- Nama tempat
- Alamat
- Rating
- Deskripsi singkat
- Kenapa cocok untuk user
`;
```

Error Boundary

```
// frontend/src/components/ErrorBoundary.tsx
import { Component, ReactNode } from 'react';

interface Props {
  children: ReactNode;
  fallback?: ReactNode;
}

interface State {
  hasError: boolean;
  error?: Error;
}
```

```

}

export class ErrorBoundary extends Component<Props, State> {
  constructor(props: Props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError(error: Error): State {
    return { hasError: true, error };
  }

  render() {
    if (this.state.hasError) {
      return (
        this.props.fallback || (
          <div className="error-boundary">
            <h2>Something went wrong</h2>
            <p>{this.state.error?.message}</p>
            <button onClick={() => this.setState({ hasError: false })}>
              Try again
            </button>
          </div>
        )
      );
    }

    return this.props.children;
  }
}

```

Latihan

1. **Connect Agent** — Wiring Mastra agent ke Express route, test panggil endpoint `/api/ai/chat` dengan Postman
2. **Stream Response** — Implement streaming agent response di frontend, tampilkan token by token seperti ChatGPT
3. **Tool Integration** — Bikin minimal 2 Mastra tools (misal: cari tempat + cek cuaca), pastikan agent pake tools itu
4. **Error Boundary** — Bungkus komponen AI dengan `ErrorBoundary`, test dengan matikan backend
5. **Prompt Templates** — Bikin 2 template prompt dengan parameter dinamis, test hasil generate-nya

Sprint 4 — Polish + Deploy

Sesi: 25-26 | Durasi: 4 jam

Tujuan

- Error handling global di backend + frontend
- Responsive check (mobile/tablet/desktop)
- Environment variables untuk production
- Deploy backend ke Railway
- Deploy frontend ke Vercel
- Custom domain (opsional)
- Post-deploy testing
- CI/CD pipeline
- Persiapan presentasi + demo

Global Error Handler — Backend

```
// backend/src/middleware/errorHandler.ts
import { Request, Response, NextFunction } from 'express';

export class AppError extends Error {
  constructor(
    public statusCode: number,
    public message: string,
    public isOperational = true
  ) {
    super(message);
    Object.setPrototypeOf(this, AppError.prototype);
  }
}

export const errorHandler = (
  err: Error,
  _req: Request,
  res: Response,
  _next: NextFunction
) => {
  if (err instanceof AppError) {
    return res.status(err.statusCode).json({
      error: err.message,
    });
  }

  // Prisma error
```

```

    if (err.name === 'PrismaClientKnownRequestError') {
      return res.status(400).json({
        error: 'Database error',
      });
    }

    console.error('Unhandled error:', err);
    res.status(500).json({
      error: 'Internal server error',
    });
  });
};

```

Production Environment Variables

```

# Railway – backend
DATABASE_URL=postgresql://...
JWT_SECRET=<random-64-char>
OPENWEATHER_API_KEY=...
MASTRA_API_KEY=...
OPENAI_API_KEY=...
PORT=3001
NODE_ENV=production

# Vercel – frontend
VITE_API_URL=https://backend-production.up.railway.app/api

```

Deploy Backend ke Railway

```

# 1. Install Railway CLI
npm install -g @railway/cli

# 2. Login
railway login

# 3. Init project di root backend/
cd backend
railway init

# 4. Set environment variables
railway variables set DATABASE_URL=...
railway variables set JWT_SECRET=...

# 5. Deploy
railway up

```

```
# 6. Production URL (contoh)
# https://my-project-backend.up.railway.app
```

Deploy Frontend ke Vercel

```
# 1. Install Vercel CLI
npm install -g vercel

# 2. Deploy
cd frontend
vercel --prod

# 3. Set environment variables di Vercel dashboard
# VITE_API_URL = https://backend-production.up.railway.app/api
```

CI/CD dengan GitHub Actions

```
# .github/workflows/deploy.yml
name: Deploy

on:
  push:
    branches: [main]

jobs:
  deploy-backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with:
          node-version: 18
      - run: npm ci
        working-directory: ./backend
      - run: npm run build
        working-directory: ./backend
      - name: Deploy to Railway
        run: npx railway up
        env:
          RAILWAY_TOKEN: ${ secrets.RAILWAY_TOKEN }

  deploy-frontend:
    needs: deploy-backend
    runs-on: ubuntu-latest
    steps:
```

```

- uses: actions/checkout@v3
- uses: actions/setup-node@v3
  with:
    node-version: 18
- run: npm ci
  working-directory: ./frontend
- run: npm run build
  working-directory: ./frontend
- name: Deploy to Vercel
  run: npx vercel --prod --token=${{ secrets.VERCEL_TOKEN }}

```

Post-Deploy Testing

```

// backend/tests/api.test.ts
import request from 'supertest';
import app from '../src/index';

describe('API Tests', () => {
  let token: string;

  test('POST /api/auth/register - success', async () => {
    const res = await request(app)
      .post('/api/auth/register')
      .send({ email: 'test@test.com', password: 'password123' });

    expect(res.status).toBe(201);
    expect(res.body.token).toBeDefined();
    token = res.body.token;
  });

  test('GET /api/trips - harusnya array kosong', async () => {
    const res = await request(app)
      .get('/api/trips')
      .set('Authorization', `Bearer ${token}`);

    expect(res.status).toBe(200);
    expect(Array.isArray(res.body)).toBe(true);
  });

  test('GET /api/health - server hidup', async () => {
    const res = await request(app).get('/api/health');
    expect(res.status).toBe(200);
    expect(res.body.status).toBe('ok');
  });
});

```

Responsive Check

Periksa di Chrome DevTools device toolbar:

- ☐ Mobile (375px)
- ☐ Tablet (768px)
- ☐ Desktop (1280px+)

Checklist Final

- ☐ Semua CRUD endpoint jalan
- ☐ Auth: register, login, logout, protected routes
- ☐ AI agent bisa diakses dari frontend
- ☐ Minimal 2 tools Mastra terintegrasi
- ☐ Error handling global backend
- ☐ Error boundary frontend
- ☐ Responsive di mobile + desktop
- ☐ Deployed backend di Railway
- ☐ Deployed frontend di Vercel
- ☐ README.md lengkap (nama, stack, cara run, link deploy)
- ☐ PROMPT-LOG.md berisi prompt AI yang dipakai

Persiapan Presentasi

Struktur presentasi 5 menit:

1. (30 detik) – Nama, proyek, stack
2. (1 menit) – Demo fitur utama (CRUD)
3. (1 menit) – Demo AI integration (agent + tools)
4. (1 menit) – Arsitektur (ERD, folder structure, data flow)
5. (1 menit) – Deployment + link live
6. (30 detik) – Closing + Q&A

Deployment Checklist (printable)

- ☐ Backend hidup di Railway GET /api/health → 200
- ☐ Frontend hidup di Vercel (domain vercel.app)
- ☐ CORS hanya allow frontend domain
- ☐ Environment variables production terisi
- ☐ Database production terhubung
- ☐ Build tidak error (npm run build)
- ☐ README update dengan link deploy

Latihan

1. **Fix Bug** — Inject minimal 1 bug di kode, minta teman find & fix. Dokumentasi bug di PROMPT-LOG.md
2. **Deploy Backend** — Deploy API ke Railway, pastikan /api/health return 200 dari public URL
3. **Deploy Frontend** — Deploy frontend ke Vercel, pastikan bisa komunikasi dengan backend production
4. **CI/CD** — Setup GitHub Actions workflow untuk auto-deploy setiap push ke main
5. **Presentasi** — Bikin slide sederhana (5 slide), latihan presentasi 5 menit, record screen + suara